```
# STEP 1: Install Required Libraries
# ========================================
!pip install -q transformers datasets accelerate

# ========================================
# STEP 2: Imports
# ========================================
import torch
from transformers import AutoTokenizer, AutoModelForSeq2SeqLM
from datasets import load_dataset
import re

device = "cuda" if torch.cuda.is_available() else "cpu"
print("Using device:", device)

# ========================================
# STEP 3: Load GSM8K Dataset
# ========================================
print("Loading GSM8K dataset...")
dataset = load_dataset("gsm8k", "main")

# small subset for testing
data = dataset["test"].select(range(10))

# ========================================
# STEP 4: Load Model
# ========================================
print("Loading model...")
model_name = "google/flan-t5-base"

tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModelForSeq2SeqLM.from_pretrained(model_name).to(device)

# ========================================
# STEP 5: Helper Functions
# ========================================

def extract_number(text):
    numbers = re.findall(r"[-+]?\d*\.\d+|\d+", text)
    return numbers[-1] if numbers else None


def generate_answer(question):
    prompt = f"Solve this math problem step by step:\n{question}"
    inputs = tokenizer(prompt, return_tensors="pt", truncation=True).to(device)

    outputs = model.generate(
        **inputs,
        max_length=128
    )

    decoded = tokenizer.decode(outputs[0], skip_special_tokens=True)
    return decoded

# ========================================
# STEP 6: Evaluation
# ========================================

def evaluate_model(data):
    correct = 0
    hallucinations = 0
    total = len(data)

    for i in range(total):
        question = data[i]["question"]
        true_answer = extract_number(data[i]["answer"])

        print("\n==================================")
        print(f"Question {i+1}")
        print("Q:", question)

        output = generate_answer(question)
        pred_answer = extract_number(output)
```

```
            print("Model Output:", output)
            print("Predicted:", pred_answer)
            print("True:", true_answer)

            if pred_answer == true_answer:
                correct += 1
            else:
                hallucinations += 1

        accuracy = correct / total
        hallucination_rate = hallucinations / total

        print("\n==================================")
        print("FINAL RESULTS")
        print("Total:", total)
        print("Correct:", correct)
        print("Wrong:", hallucinations)
        print("Accuracy:", round(accuracy, 4))
        print("Hallucination Rate:", round(hallucination_rate, 4))

        return accuracy, hallucination_rate

# ========================================
# STEP 7: Run
# ========================================

accuracy, hallucination_rate = evaluate_model(data)
```